

3D Mesh Quality Control

Автоматическое выявление дефектов 3D-моделей для обучения генеративных моделей

Решение. 6 фиксированных рендеров меша → замороженный DINOv3 (ViT-S+ 28.7M и ViT-B 85.6M) → 398 токенов + 27 чисел геометрии → обучаемый attentive probe (7.7M параметров) предсказывает 10 независимых дефектов и флаг quality; пороги подобраны прямо по метрике соревнования.

Метрика: $10 \cdot F1(\text{quality}) + 10 \cdot F1_w(\text{artefacts})$ из $20 \cdot \text{OOF}$ **13.68 / 20** · публичный LB **13.86 / 20**
(F1 quality 0.684, F1_w артефактов 0.685) — на 3.2 балла выше признаков геометрии (10.52) и на 0.23 выше одиночного DINOv3-s (13.45)

Задача и данные

Дано (на объект, item_id):

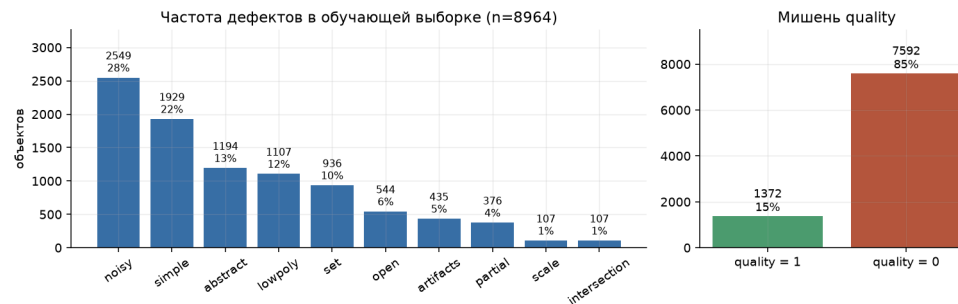
- один PNG-коллаж 1536×1024 — 6 рендеров: 4 азимута (az0/90/180/270) + вид сверху и снизу (тайлы 512×512);
- .npz с мешем: вершины и грани;
- разметка: 10 бинарных дефектов + флаг quality.

Размеры: 8964 обучающих объекта, 669 тестовых.

Метрика:

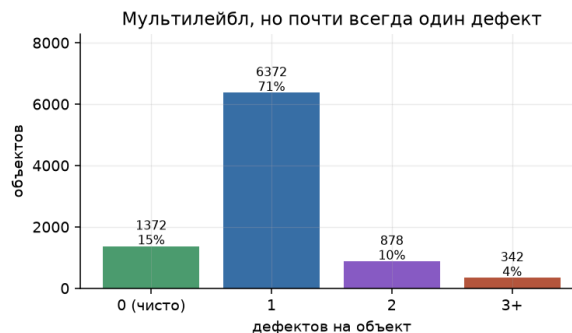
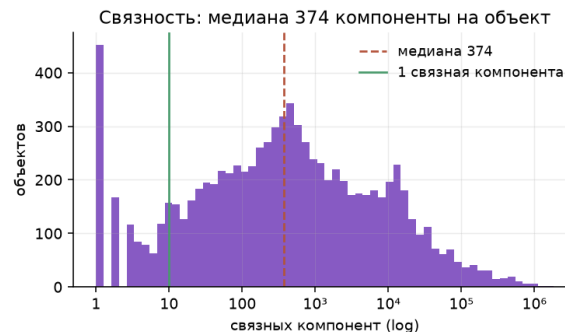
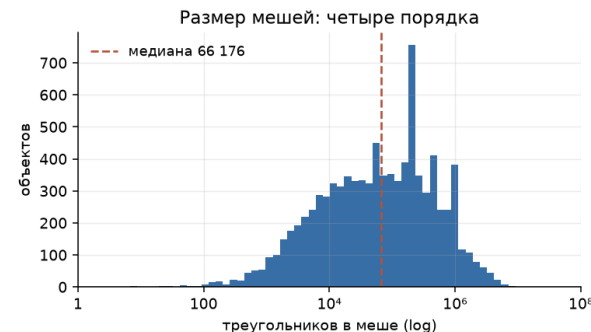
$$10 \cdot F1(\text{quality}) + 10 \cdot F1_{\text{weighted}}(\text{defects})$$

Классы сильно несбалансированы: noisy встречается у 28% объектов, intersection и scale — у 1.2% (по 107 объектов). Ошибка на самом частом классе весит почти в 24 раза больше, чем на самом редком.



Частота дефектов в train (n=8964) и целевой флаг quality: «чистых» объектов всего 15%.

Данные: меши — не поверхности, а «суп из треугольников»



- **Размер:** от 4 до $2.6 \cdot 10^7$ треугольников (медиана 66 176) — четыре порядка. Признаки обязаны быть устойчивы к масштабу и плотности.
- **Связность:** медиана — 374 компоненты на объект, у 93% объектов их больше одной. Перед любой топологией нужна сварка вершин.
- **Масштаб:** диагональ bbox разбросана на девять порядков, меши не центрированы. Рендеры при этом каноничны (объект вписан в unit cube) — абсолютный размер виден только в геометрии, отсюда токен [GEOM].
- **Разметка:** 71% объектов имеют ровно один дефект — задача почти одноклассовая, несмотря на мультилейбл-формат.

Наблюдение 1: `quality` выводится из дефектов

Проверка на всех 8964 объектах:

```
quality == 1 <=> все 10 дефектов == 0
```

Это правило разметки, а не совпадение. Поэтому отдельная модель для quality не нужна:

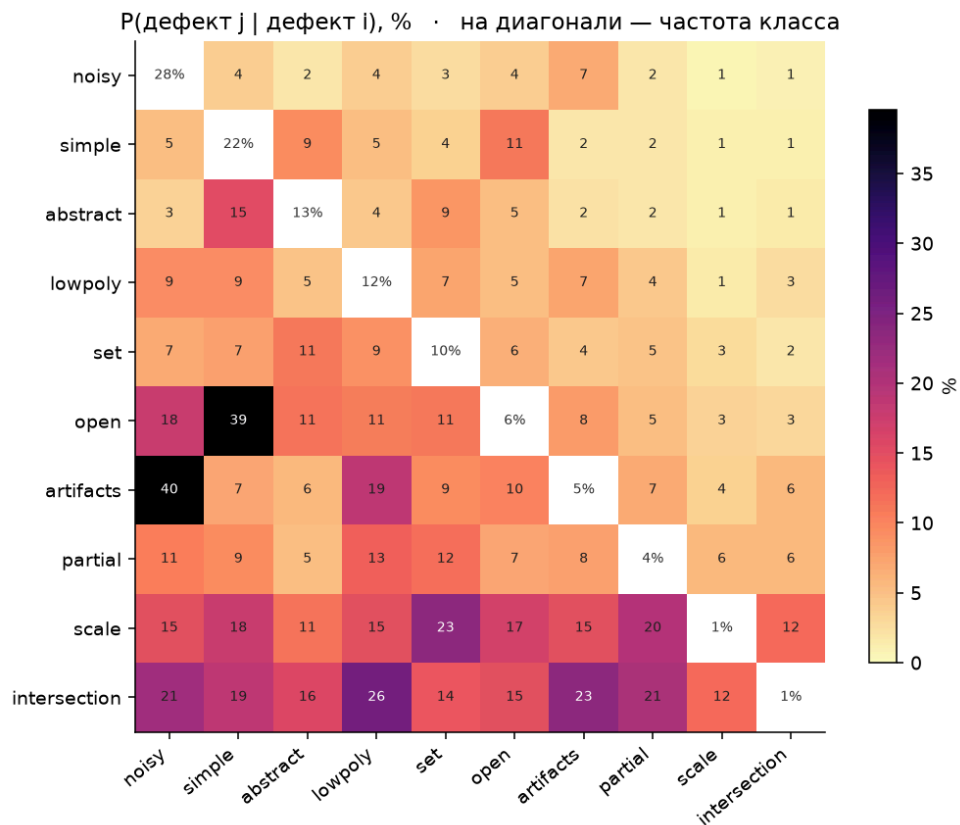
```
quality_hat = int(sum(defects_hat) == 0)
```

Что это даёт:

- 10 задач вместо 11 — меньше параметров и поводов для ошибки;
- F1(quality) становится **согласованной** метрикой: её улучшает уменьшение ложных срабатываний дефектов, а не подстройка флага.

Дефекты при этом коррелируют: open → simple 39%, artifacts → noisy 40%, intersection → lowpoly 26% — редкие классы почти всегда идут в паре с частым. Поэтому независимые сигмоиды — правильный выбор (softmax заставлял бы выбирать один класс).

F1(quality) — ровно половина метрики: 10 из 20 баллов.



Условная вероятность $P(\text{дефект } j \mid \text{дефект } i)$; на диагонали — частота класса.

Наблюдение 2: решают 10 порогов, а не 10 вероятностей

Вес класса в `F1_weighted` жёстко задан его поддержкой:

класс	вес
noisy	27.5%
simple	20.8%
abstract	12.9%
lowpoly	11.9%
set	10.1%
open	5.9%
artifacts	4.7%
partial	4.1%
intersection	1.2%
scale	1.2%

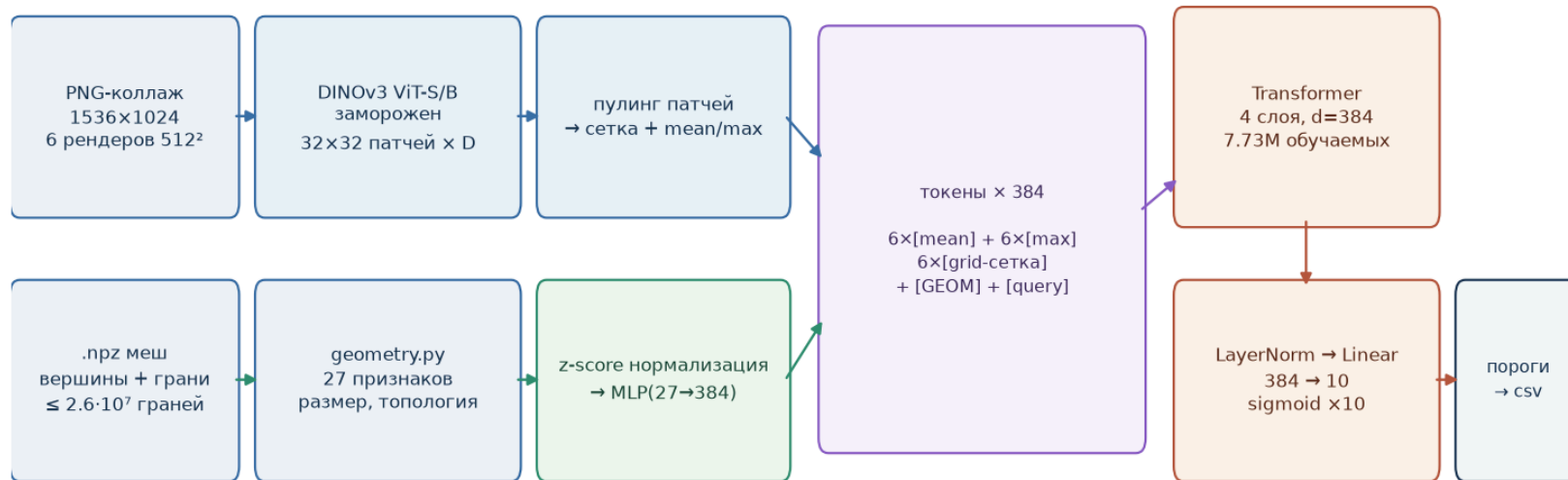
Главный вывод. Модель выдаёт вероятности, а баллы начисляются за бинарные предсказания. Поэтому итог решает порог каждого из десяти детекторов, а не качество ранжирования.

- у редкого класса (107 положительных) оптимум смещён к высокому порогу: пропуск дешевле ложного срабатывания;
- у частого — наоборот;
- `quality` связывает все десять порогов: любой ложный дефект убирает объект из «чистых», а их всего 15%.

Практический вывод: обучаем десять независимых сигмoids, а решение принимаем по десяти порогам, подобранным **совместно** прямо по метрике соревнования (координатный подъём, слайд 11).

Решение: замороженные признаки + лёгкая обучаемая агрегация

Замороженные признаки (обучается только проб) + дешёвая обучаемая агрегация



quality = «ни одного дефекта выше порога» — правило разметки подтверждено на всех 8964 объектах

1. Признаки без обучения. DINOv3 заморожен: 8964 объектов мало для обучения ViT. В кэш пишем только сетку 8x8 и mean/max по тайлу, а не полный 32x32 — признаки читаются с диска за минуты.

2. Внимание вместо классификатора. 398 токенов (визуальные + [GEOM]) проходят через 4 слоя трансформера; обучаемый [query] собирает улики по всем ракурсам в один вектор.

3. Пороги и слияние. Два проба (s и b) усредняются по вероятностям; пороги подбираются на OOF по метрике соревнования. Обучаемых параметров — 7.7М из 36.4М.

Признаки: 27 чисел геометрии прямо из меша

Что это. 27 чисел из вершин и граней .npz, без изображений:

- размеры: bbox_dx/dy/dz, диагональ, объём, площадь;
- форма: вытянутость, компактность, доля объёма от bbox;
- топология: вершины, грани, рёбра, компоненты связности, дырки (эйлерова характеристика);
- плотности: вершин и граней на единицу площади/объёма, средние размеры граней и рёбер.

Как оцениваем: одна модель градиентного бустинга (HistGB) на каждый дефект. Результат — **10.52 / 20 (OOF)**: базовая линия для всего остального.

Геометрия ловит структурные дефекты: noisy F1 0.78 (шум — это выбросы вершин), simple 0.67, abstract 0.53, lowpoly 0.53.

Но там, где нужно **увидеть** объект, она слепа: intersection 0.00, scale 0.10, artifacts 0.13, open 0.20.

Вывод: геометрию оставляем как отдельный токен [GEOM] — она несёт абсолютный размер, которого нет в каноническом рендере, — но ставку делаем на визуальные признаки.

Признаки: рендеры через замороженный DINOv3

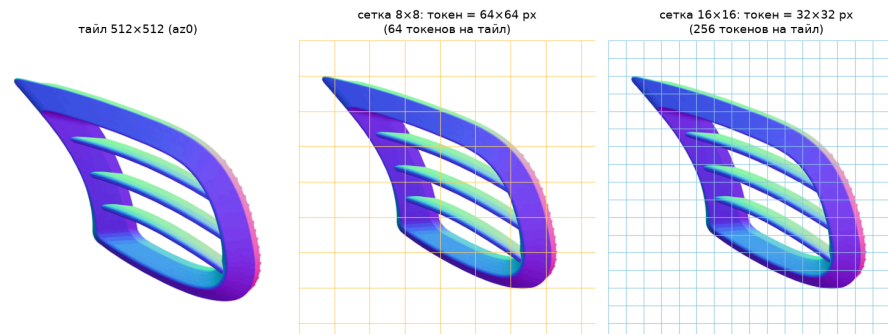
Почему заморозка и почему DINOv3. 8964 объектов — слишком мало для обучения ViT с нуля; DINOv3 — self-supervised модель с лучшими плотными признаками (NAVI 64.4), а схема «патчи → несколько слоёв внимания → query» — стандарт для замороженных энкодеров (V-JEPA 2, iBOT). Для будущего меш-энкодера задел — LeJEPA/SIGReg (финальный этап).

Токенизация. Коллаж → 6 тайлов 512×512 → ViT-S+/16 (28.7M) или ViT-B/16 (85.6M): сетка 32×32 патч-токенов, $d=384$ или 768. Позиции — RoPE, нормализация — статистика ImageNet.

Что кладём в кэш (fp16):

- сетку 8×8 — токен = среднее блока 4×4 патчей (область 64×64 px), положение сохраняется;
- mean и max по всем патчам тайла — глобальное описание ракурса.

Узкое место — не ёмкость: ViT-B (768-d) **слабее** ViT-S при том же пробе. Потолок задают данные и разрешение признаков.



Цена компромисса: токен сетки 8×8 покрывает 64×64 px, мелкая локальная улика размывается усреднением. Сетку 16×16 проверили на полной CV — выигрыша нет (слайд «Аблации»).

Архитектура: attentive probe по 398 токенам

Вход трансформера — 398 токенов, все $d=384$:

- 6 **mean** — тайл, усреднённый по патчам
- 6 **max** — тот же тайл, свёрнутый максимумом: ловит редкие яркие признаки
- 384 **grid** — сетка 8×8 на тайл: хранит, где в тайле признак
- 1 **[GEOM]** — 27 чисел (z-score) $\rightarrow$ MLP(27 $\rightarrow$ 384)
- 1 **[query]** — обучаемый агрегатор; его выход $\rightarrow$ 10 sigmoid-логитов

Эмбединги тайла (6 рендеров) и типа (mean/max/grid) добавляются **после** проекции — иначе 768-мерный энкодер не сложил бы их с 384-мерными признаками.

Почему трансформер, а не линейная голова. Нужно сопоставлять улики между ракурсами («деталь видна на az_0 , исчезает на az_{180} ») и связывать [GEOM] с визуальными токенами. 4 слоя, 6 голов, $d=384$ — **7.73M** обучаемых параметров (чекпоинт 31 МБ), обучение — минуты на GPU.

Абляция ёмкости. depth 2 даёт -0.39 балла относительно depth 4: четыре слоя нужны, а больший энкодер (b, l) — нет (см. «Результаты»).

Обучение: 5-фолдовая CV и один рецепт для всех прогонов

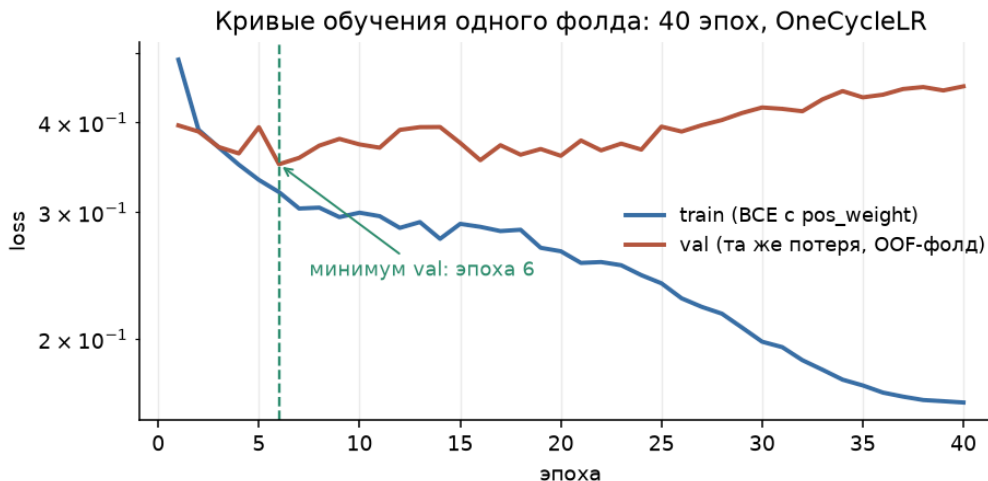
Протокол:

- 5 фолдов ($seed=0$), обучение на 4/5; OOF-предсказания — для порогов и всех честных метрик;
- после CV — refit на всех данных (несколько seed-состояний, затем усредняются).

Рецепт:

эпох / batch	40 / 128
оптимизатор	AdamW, lr 1e-3, wd 0.05
расписание	OneCycleLR, pct_start 0.1
loss	BCEWithLogits, pos_weight = (neg/pos)^0.5
точность	bf16 autocast, clip 1.0
время	4.5 мин / фолд (RTX 5070)

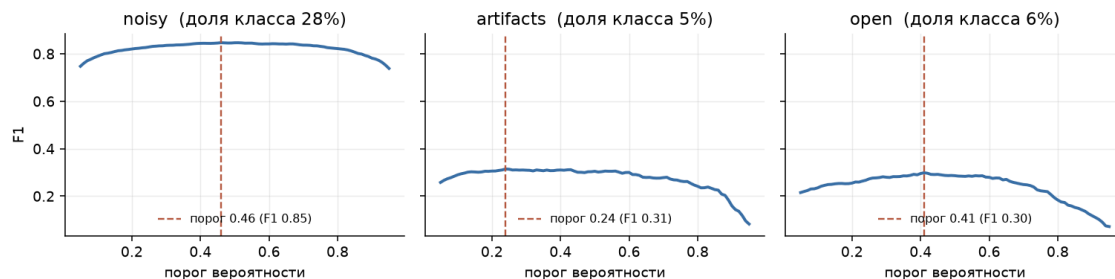
10 эпох дают тот же OOF, что 40 (13.47 против 13.45): бюджет обучения переоценён в 4 раза — это запас времени финального этапа.



Кривые одного фолда (train/val — одна и та же потеря с pos_weight). Train падает до 0.16, val — минимум 0.35 к 6-й эпохе, затем растёт: по лог-лоссу проб переобучается. Но метрику решают пороги по OOF, поэтому глубокое дообучение не нужно.

Пороги: координатный подъём прямо по метрике

Порог решает всё: F1(класса) как функция порога (OOF, DINOv3-s + проб)



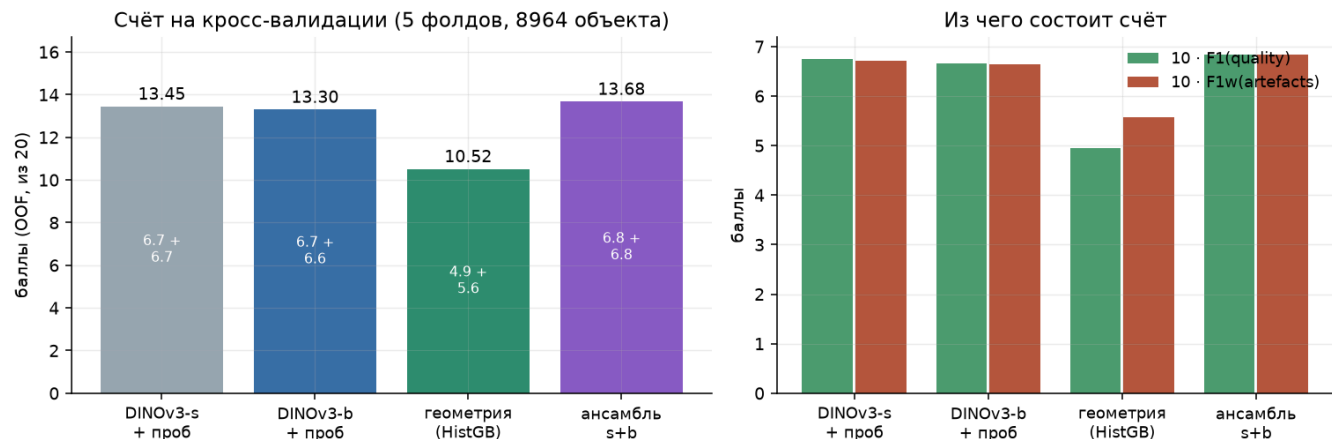
Как: берём OOF-вероятности и жадно перебираем порог каждого класса (30 значений) при фиксированных остальных, повторяем 3 прохода, пока счёт не перестанет расти. Целевая функция — сама метрика $10 \cdot F1(\text{quality}) + 10 \cdot F1_w(\text{defects})$.

Что видно на кривых:

- у noisy (28% объектов) широкое плато — порог почти не важен;
- у open и artifacts оптимум смещён, а пик ниже: редкие локальные дефекты дорого ловить «в лоб»;
- F1(quality) зависит от порогов **всех** классов: один ложный дефект убирает объект из «чистых»;
- у пробов s и b разные шкалы вероятностей — при слиянии пороги подбираются заново на усреднённых OOF.

Подбор порогов даёт $\approx +0.4$ балла относительно порога 0.5 — больше, чем любая аблация рецепта.

Результаты

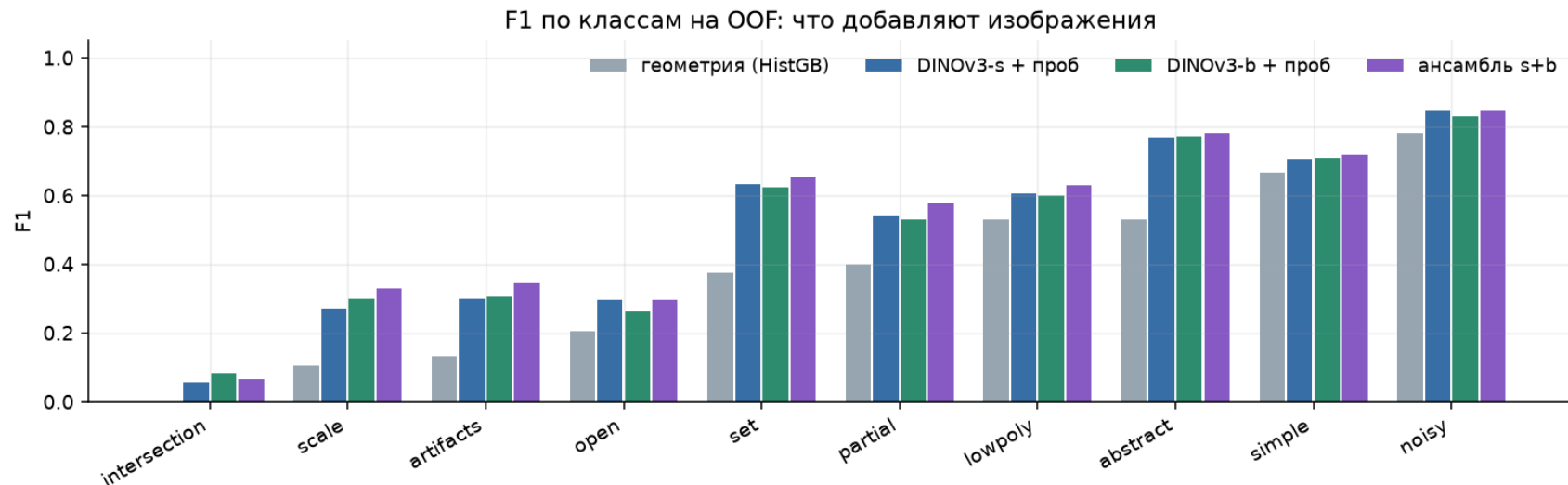


вариант	OOF / 20	F1 quality	F1_w артефактов
геометрия, HistGB (27 чисел)	10.52	0.495	0.557
HistGB на image-признаках (PCA-256)	11.72	0.555	0.617
HistGB: рендеры + геометрия	11.75	0.556	0.619
DINOv3-s + attentive probe	13.45	0.675	0.671
DINOv3-b + attentive probe	13.30	0.666	0.664
слияние s+b (отправлено)	13.68	0.684	0.685

+2.9 балла дают визуальные признаки (10.52 → 13.45), ещё +0.23 — слияние двух энкодеров. Большой энкодер сам по себе слабее: потолок задают масштаб данных и разрешение признаков, а не размер модели.

Публичный LB: **13.86 / 20** у слияния s+b — выше OOF 13.68. Первая версия v1 (до переобучения проба) набрала 12.53 (место 65): разрыв объяснялся протухшим чек-пойнтом, а не сложностью теста.

Что дают рендеры: разбор по классам и бюджет очков



Выигрыш там, где нужен вид: abstract 0.53→0.78, set 0.37→0.65, lowpoly 0.53→0.63, partial 0.40→0.58, noisy 0.78→0.85.

Остаются тяжёлыми: intersection 0.06, open 0.30, scale 0.33, artifacts 0.34 — редкие и локальные дефекты, где геометрия тоже слаба.

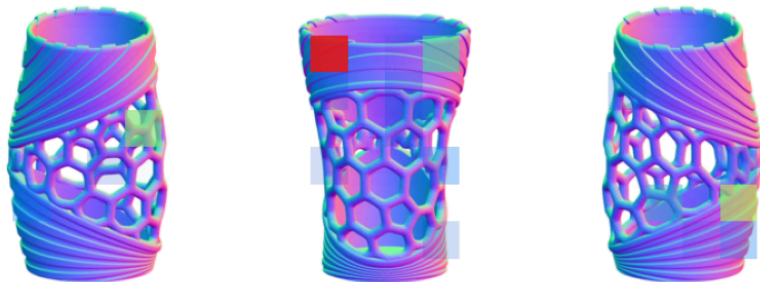
Куда уходят баллы (бюджет $10 \times \text{вес} \times (1-F1)$): simple -0.59, lowpoly -0.44, noisy -0.42, open -0.41. Три самых частых класса теряют 1.45 из 3.16 — больше, чем все редкие вместе (0.67).

Рычаг — аккуратность на частых классах и ложные срабатывания, а не только редкие классы.

Структура ошибок и разбор кейсов

Успехи и ошибки: где модель «смотрит» (виды az0/az90/az180, градиентная атрибуция, проб s)

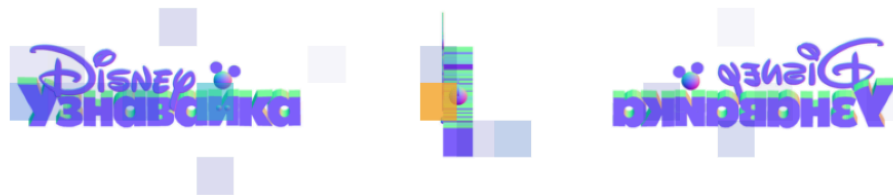
FN: `artifacts` есть, модель промолчала
9048f70b · p(artifacts) = 0.04 · истина: artifacts



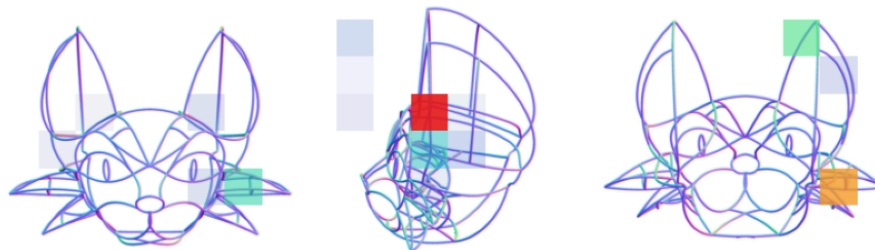
тест: уверенный `artifacts` p=0.95
722cb749 · p(artifacts) = 0.96 · тест: разметки нет, показана только уверенность



FP: чистый объект назван `abstract`
e6d23a92 · p(abstract) = 0.95 · истина: чисто



тест: уверенный `open` p=0.94
40a01d37 · p(open) = 0.93 · тест: разметки нет, показана только уверенность



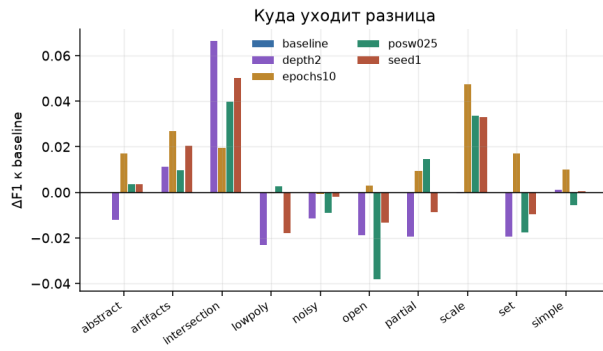
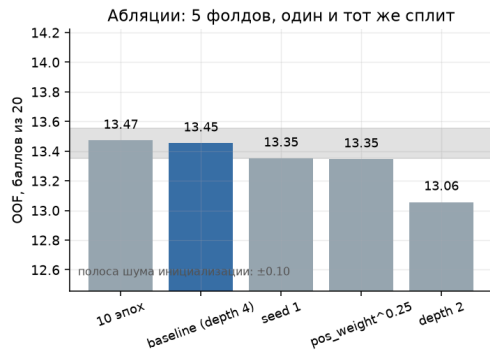
FN: у перфорированной вазы artifacts=1, модель смотрит в одну ячейку и даёт p=0.04 — редкий дефект (435 примеров) без похожих положительных.

FP: 3D-логотип размечен как «чистый», но abstract по условию — это тексты и логотипы: модель настаивает на 0.95 и, вероятно, права. Шумная разметка.

Тест без разметки: уверенные предсказания (p=0.96 artifacts, 0.94 open) с атрибуцией на рендерах — именно эти панели строит интерпретационный ноутбук.

Ошибки зеркальны по классам: open даёт 578 ложных срабатываний на 773 прогноза (precision 0.25), noisy пропускает 435 при 2549 объектах. Для quality: 958 «чистых» удержаны, на 414 чистых сработал ложный дефект, у 473 дефектных пропущены дефекты.

Аблации: шумовая полка ± 0.1 и проверка разрешения



вариант

baseline (depth 4, grid 8)
 depth 2
 10 эпох вместо 40
 pos_weight^(1/4)
 тот же рецепт, seed 1
 grid 8, mean+max
 grid 4, mean+max

OOF вывод

13.45 отправленный рецепт
 13.06 -0.39: четыре слоя нужны
 13.47 +0.02: шум, но 4× быстрее
 13.35 -0.11: граница шума
 13.35 **шумовая полка $\approx \pm 0.1$**
 13.42 полная CV: в пределах шума
 13.43 полная CV: в пределах шума

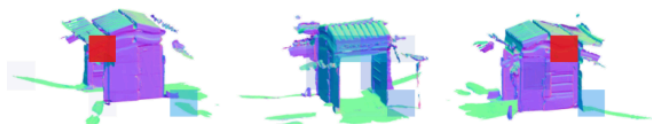
Single-fold «скрины» обманывают: на одном фолде grid 8 + mean+max дал +0.24, на полной CV — минус. Считать только полной CV.

Итог: значим лишь depth 2; разрешение и пулинг счёт не двигают, поэтому в рефакторинге оставлен один рецепт (8×8 + mean/max).

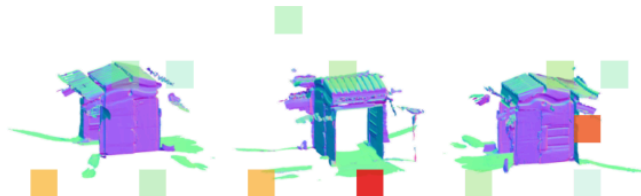
Объяснение решения: три метода на одной улике

Три метода объяснения одной и той же улики: `artifacts`, $p = 0.92 \cdot f08554bd$ (виды az0, az90, az180; цвет ячейки = вклад)

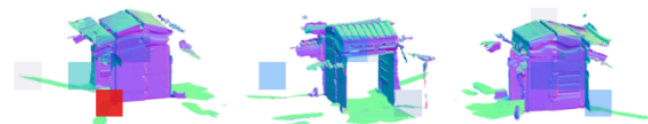
градиент $\times$ активация



окклюзия ячейки (Δlogit)



внимание query $\rightarrow$ ячейки



Градиент $\times$ активация: $|\text{grad} \times \text{activation}|$ логита по ячейкам, top-15%. Быстро, но шумно — зависит от одной точки.

Окклюзия ячейки: обнуляем признак ячейки, смотрим Δ логита. Дороже в 384 раза, зато причинное свидетельство.

Внимание [query]: среднее attention последнего слоя от query к ячейкам — без градиентов.

Все три метода указывают на одни и те же 1–2 ячейки из 384 — это главный аргумент за более мелкую сетку.

Leave-one-view-out: улики живут в азимутах, top/bottom почти не нужны. Для open вид az180 **подавлял** логит (-4.4 при удалении) — источник пропусков.

Воспроизводимость: Colab без ручных правок

Ноутбук `colab_solution.ipynb`:

- ставит зависимости и сам скачивает данные из условия задачи (26.6 ГиБ `AIC_data.tar`, распаковка только PNG);
- извлекает признаки DINOv3 и считает вероятности тем же кодом `src/mesh_quality/`, что и локально;
- сверяет `submission.csv` с **замороженной** копией вероятностей (`reference_probs`, md5 в манифесте), затем считает OOF-метрику;
- фиксирует `SEED = 42`.

Бюджет Colab (T4): признаки 669 объектов $\approx$ 1 мин, инференс и сверка $\approx$ 2 мин, обучение 5 фолдов $\approx$ 25–40 мин.

Что нашли в процессе — три дефекта пайплайна:

- чекпоинт, обученный до фикса сборки токенов, разошёлся со своим OOF (v1: LB 12.53 $\rightarrow$ после переобучения 13.86). Артефакт обязан хранить ревизию кода;
- `submissions.csv` писался небезопасно: неудачный `append` стирал историю — теперь запись атомарная;
- проверка сабмита сравнивалась с файлом, который сам `predict` перезаписывает, — эталон вынесен в неизменяемую копию.

Два «загадочных» расхождения оказались багами пайплайна, а не свойствами данных: сначала проверяем артефакты, потом обвиняем тест.

Выводы и план финального этапа

Что сделано:

- quality выводится из дефектов (правило подтверждено на 8964 объектах);
- геометрия HistGB — базовая линия 10.52;
- замороженный DINOv3 + attentive probe — **13.45** OOF;
- слияние s+b — **13.68** OOF и **13.86** на публичном LB (лучший сабмит);
- пороги подобраны прямо по метрике; аблации и журнал — в cache/ablations/ и submissions.csv;
- воспроизведение в Colab одной кнопкой.

План финального этапа (по ожидаемой пользе):

1. **Диагностика:** in-sample vs OOF по классам, калибровка вероятностей и взвешенное слияние s+b (у энкодеров разные шкалы).
2. **Меш-энкодер:** патчи поверхности + per-patch статистики отдельными токенами; JEPA/SIGReg — только если статистики проходят гейт (+0.2 к 13.45). Работа уже начата.
3. **Ёмкость головы** (depth 8 / d=512) — последней: depth 2 хуже depth 4, но больший энкодер не помогает.
4. **Дешёвые резервы:** 10 эпох вместо 40 (4× больше экспериментов), аугментации рендеров, seed-усреднение.

Разрешение признаков проверено на полной CV и отклонено — не тратим на него финальный этап.

Приложение: запуск и технические факты

CLI (из корня репозитория, единое окружение):

```
devenv shell -- uv run python main/mesh-quality/main.py features --model s
devenv shell -- uv run python main/mesh-quality/main.py train --folds 5
devenv shell -- uv run python main/mesh-quality/main.py predict --models s,b
devenv shell -- uv run python main/mesh-quality/main.py score --submission submission.csv
```

Аблации и фигуры:

```
uv run python main/mesh-quality/tools/hyperparams.py --list
uv run python main/mesh-quality/tools/analysis_figures.py --cpu-only
```

Артефакты: submissions.csv (журнал сабмитов), submission.csv (лучший), notebooks/ (colab_solution, interpretation), presentation/ (эта презентация).

энкодеры (заморожены)	DINOv3 ViT-S+ 28.7M и ViT-B 85.6M
обучаемая часть	7.73M на энкодер, чекпоинт 31 МБ
токенов на объект	398: 6 mean + 6 max + 384 grid + [GEOM] + [query]
кэш признаков s+b	2.6 + 5.1 ГБ train, fp16
обучение 1 фолда	4.5 мин (RTX 5070)
инференс 669 объектов	20 с на энкодер
сырой архив	AIC_data.tar 26.6 ГиБ

Полный код, метрики и команды воспроизведения — в репозитории (main/mesh-quality/) и в Colab-ноутбуке решения.